



Gestão Fácil — App Mobile

^ Expo 49.0.0

React Native 0.76

license MIT

Axios HTTP

Auth JWT

Aplicativo mobile desenvolvido com **Expo/React Native** para **gestão de clientes, ativos e ordens de serviço (OS)**.
Inclui **autenticação JWT**, **controle de acesso por cliente**, **navegação fluida** (tabs/stacks) e integração com o backend [Gestão Fácil API](#).



Funcionalidades

- 🔑 **Autenticação JWT** (login, registro e persistência de sessão)
- 🏠 **Clientes** — listagem, cadastro e vínculo com ativos e OS
- ⚙️ **Ativos** — cadastro vinculado a local e cliente
- 📄 **Ordens de Serviço (OS)** — listagem global e por cliente/ativo, criação e conclusão
- 📍 **Locais** — seleção e associação em novos ativos
- 🌞 **Tema claro/escuro** dinâmico via Context API
- 🌀 **Navegação moderna** com React Navigation (Tabs + Stacks)
- 🔄 **Atualização automática** ao retornar às telas (`useFocusEffect`)



Tecnologias Utilizadas

Tecnologia	Descrição
Expo + React Native	Base do app mobile multiplataforma
React Navigation	Navegação em abas e pilhas
Axios	Cliente HTTP com interceptors JWT
AsyncStorage / SecureStore	Persistência local segura
React Context API	Controle global de autenticação, tema e dados
dotenv	Configuração de variáveis de ambiente (<code>EXPO_PUBLIC_API_URL</code>)



Requisitos

- Node.js **v18+**
- Expo CLI (`npm install -g expo-cli`)
- Backend **Gestão Fácil API** em execução (local ou remoto)

Variáveis de Ambiente

Crie um arquivo `.env` na raiz do projeto:

```
EXPO_PUBLIC_API_URL=https://seu-backend.exemplo.com
```

O `src/service/api.js` resolve automaticamente a `baseUrl` conforme a plataforma:

- Android emulador → `http://10.0.2.2:3000`
- iOS simulador → `http://localhost:3000`

Como Executar

1 Instalar dependências

```
npm install
```

2 Iniciar o servidor de desenvolvimento

```
npm start
```

3 Executar o app

- **Expo Go (QR Code)**
- **Android Studio**
- **Xcode (iOS)**

Scripts úteis (package.json):

```
npm run start  
npm run android  
npm run ios  
npm run web
```

Estrutura do Projeto

```
src/  
  routes/  
    index.js  
    stack.routes.js  
    tab.routes.js
```

```
screens/  
  Auth/ (Login, SignUp)  
  Clients/ (lista, ativos, OS, novo cliente)  
  Clients/ClientAssets/NewAsset/  
  Clients/ServiceOrders/  
  Assets/  
  Locations/  
  ServiceTypes/  
  Users/  
  Home/  
  Options/  
contexts/  
  auth/  
  clients/  
  serviceOrders/  
  theme/  
service/  
  api.js  
  tokenService.js  
  authService.js  
  clienteService.js  
  ativoService.js  
  servicoService.js  
  localService.js  
components/  
  Button.js  
  Input.js  
  ClientCard.js  
  Loading.js
```

Navegação

Stacks e Abas

Stack	Telas	Descrição
AuthStack	Login, SignUp	Controle de autenticação
AppTabs	Dashboard, Clientes, Serviços, Ativos, Mais	Navegação principal
MaisStack	Locais, Tipos de Serviço, Usuários (admin)	Módulos adicionais

Autenticação e Segurança

- `AuthContext` gerencia o estado global:

```
{ user, isAuthenticated, login, logout, checkAuth }
```

- `tokenService` armazena localmente:

- `userToken, userEmail, userRole, userId, userClientId`
- `api.js` injeta o cabeçalho `Authorization: Bearer <token>`
- Erro **401 Unauthorized** → sessão limpa e retorno automático ao login

Escopo por Cliente

Se o token JWT contiver `clientId` (ou `clienteId`), o app restringe:

- Listagens de **Cientes, Ativos e Serviços**
- Criação de OS com vínculo automático a `clienteId`

Endpoints Esperados (Backend)

Módulo	Método	Rota	Descrição
Auth	POST	<code>/auth/login</code>	Login com e-mail/senha
	POST	<code>/auth/register</code>	Registro de usuário
Cientes	CRUD	<code>/v1/clientes</code>	Gestão de clientes
Ativos	CRUD	<code>/v1/ativos?clienteId=</code>	Gestão de ativos
Serviços	CRUD	<code>/v1/servicos?clienteId=&ativoId=</code>	Ordens de serviço
Locais	GET	<code>/v1/locais</code>	Seleção de locais

Fluxos Principais

Clientes

- Listagem filtrada (por escopo de cliente do usuário)
- Cadastro com validação de campos obrigatórios

Ativos

- Lista global ou por cliente
- Cadastro requer `localId` e `clienteId`

Ordens de Serviço

- Listagem global ou filtrada (cliente/ativo)
- Criação, edição e conclusão com badges de status

Convenções e Boas Práticas

- **Services** normalizam campos do backend → frontend (`numero_serie` → `numeroSerie`)
- Evite usar dados brutos; use campos mapeados

- Utilize `useFocusEffect` para recarregar dados ao voltar às telas
 - Centralize requisições no `api.js` com interceptors e timeout
-

Roadmap

- ☐ Paginação e busca **server-side**
 - ☐ Telas de **detalhe** para Ativos e OS
 - ☐ CRUD completo de **Tipos de Serviço** e **Usuários (admin)**
 - ☐ Armazenamento seguro com **SecureStore (Expo)**
 - ☐ Upload de imagens (ativos/usuários)
-

Autor

Érico de Freitas Neto

📍 Líder Técnico em Sistemas Embarcados e Videomonitoramento

🔗 [GitHub – EricofreitasNeto](#)

✉ erico@exemplo.com

🌐 [LinkedIn – Erico de Freitas Neto](#)

Licença

Este projeto está sob a licença **MIT**.

Veja o arquivo [LICENSE](#) para mais detalhes.

Preview

 React Navigation **Tabs + Stacks**

Theme **Claro/Escuro**

API **RESTful**

 Repositório complementar:

 [Gestão Fácil Backend \(API REST\)](#)
